

Universal Agent Host

UI Variant Showcase & Comparative Analysis

This document presents every UI variant built for the Universal Agent Host project. Each variant explores a different information architecture, layout approach, and interaction model for managing AI agents in software development workflows.

14 variants across three categories: 8 main operational views, 4 startup/landing pages, and 2 workflow-oriented task managers built on a new type system designed around real development workflows.

92 screenshot tests verify visual consistency across all variants.

Global Navigation Bar



The shared navigation bar connects all 14 variants, grouped into Main, Startup, and Workflow sections.

Table of Contents

Main Variants

1. AgentOS — *Native Desktop IDE for AI Agent Orchestration*
2. Hive — *Agent-Centric Kanban with Trust Level Management*
3. Pipeline — *Stage-Gate Workflow with Human Review Checkpoints*
4. Nerve Center — *Real-Time Monitoring & Intervention Dashboard*
5. Mosaic — *Strategic Overview Radiator with 8 Information Cards*
6. Command Center — *Unified 5-Panel Operational Command Interface*
7. Flow — *Narrative Vertical Scroll with Sequential Information Flow*
8. Spatial — *Graph-Based Canvas for Entity Relationship Mapping*

Startup Variants

9. Startup: Chat — *Chat-First Interface with Quick Action Chips*
10. Startup: Dashboard — *Status Overview with Quick Actions*
11. Startup: Command — *Raycast/Spotlight-Style Command Palette*
12. Startup: Brief — *Mission Briefing with Chat Sidebar*

Workflow Variants

13. Ops — *Focused Task Manager Bridging Dev Workflow*
14. Kanban — *Workflow Board with Issue Lifecycle Columns*

Appendix A: Comparative Analysis

Appendix B: Information Architecture Dimensions

Appendix C: Verification Pipeline Deep Dive

MAIN VARIANTS

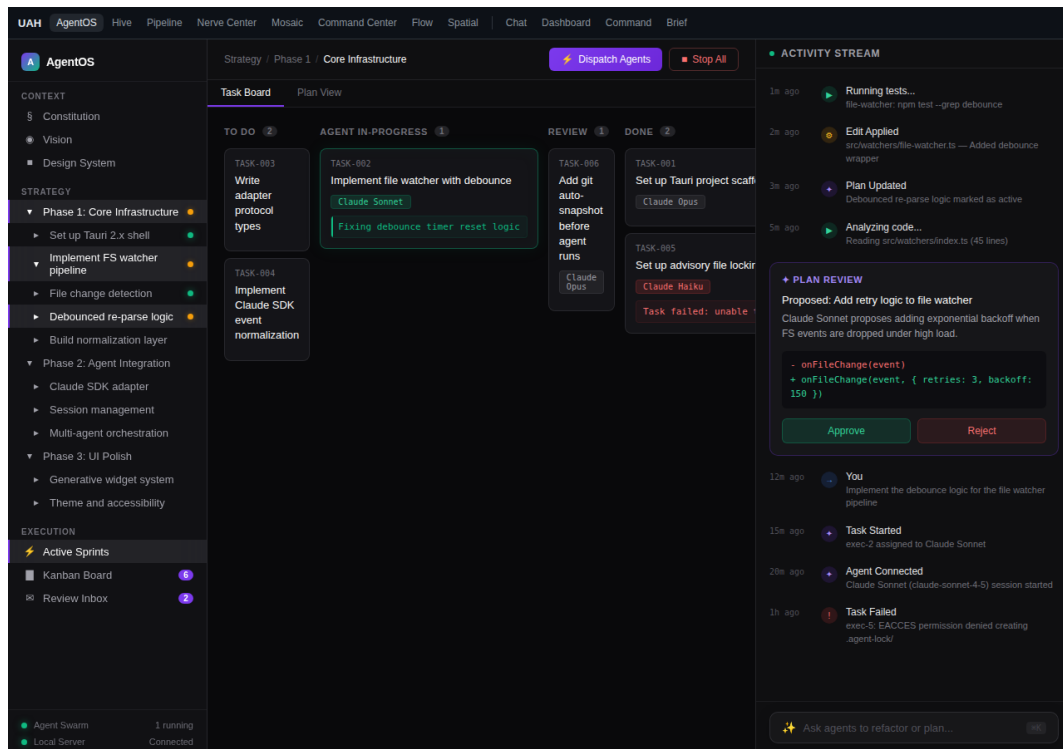
Eight operational views exploring different information architectures for managing AI agents. Each uses the original data model based on abstract primitives: Context, Strategy, Execution, and Verification.

1. AgentOS

Native Desktop IDE for AI Agent Orchestration

Layout

Three-column sidebar + main content area with tabbed views



Full Page — Task Board

Description

AgentOS brings a native desktop IDE feel to agent management. A semantic sidebar navigator acts as a "lens" for viewing different contexts within the strategy tree, while the main area hosts a task board with drag-and-drop columns and a plan view for hierarchical strategy decomposition. The bottom console provides real-time agent log streaming with a magic input bar for natural language commands.

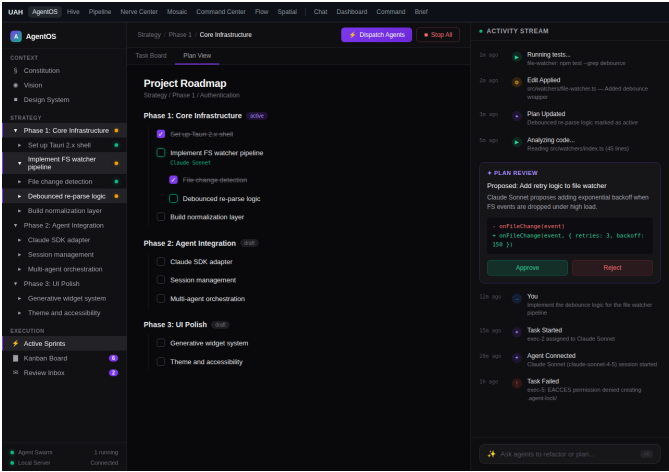
Design Reasoning

Designed for power users who need simultaneous visibility into strategy, execution, and agent output. The IDE metaphor is familiar to developers and supports deep focus work. The sidebar-main-console layout maximizes information density while maintaining clear separation of concerns.

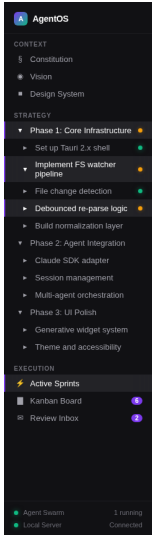
Key Components

- Semantic sidebar with strategy/context/agent navigation
- Task board with queued/running/review/done columns
- Plan view with hierarchical strategy tree
- Real-time agent console with streaming logs
- Magic input bar for natural language commands
- Generative widget for AI-powered suggestions

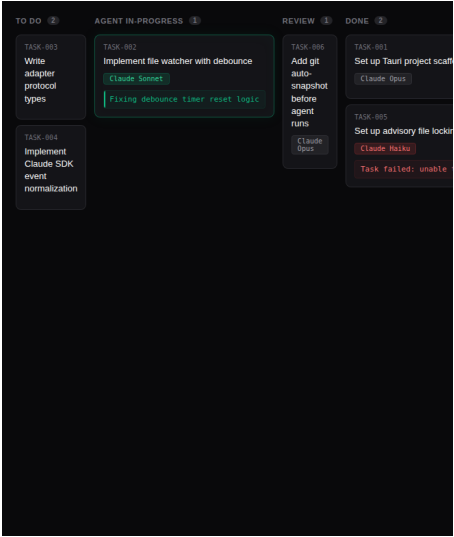
AgentOS — Component Details



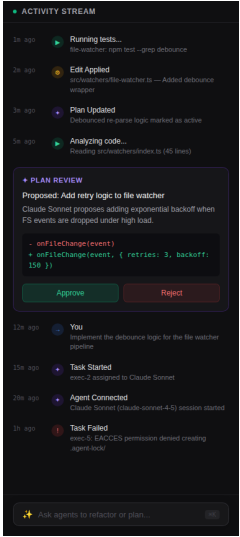
Full Page — Plan View



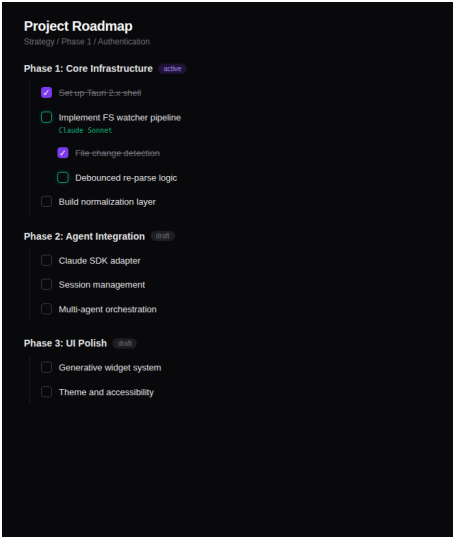
Sidebar Navigator



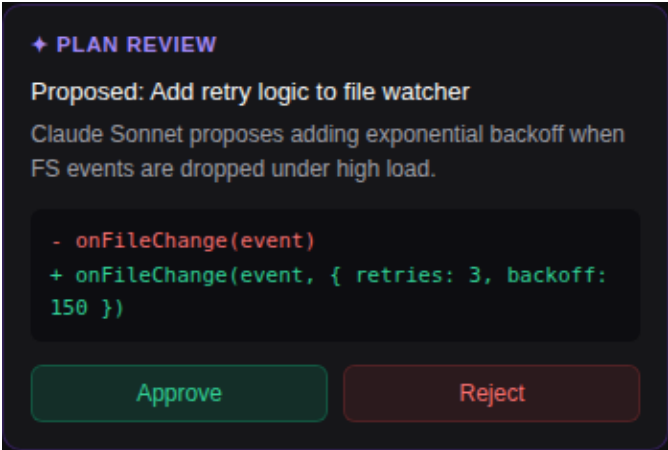
Task Board



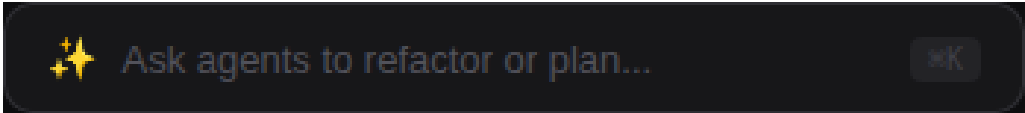
Agent Console



Plan View



Generative Widget



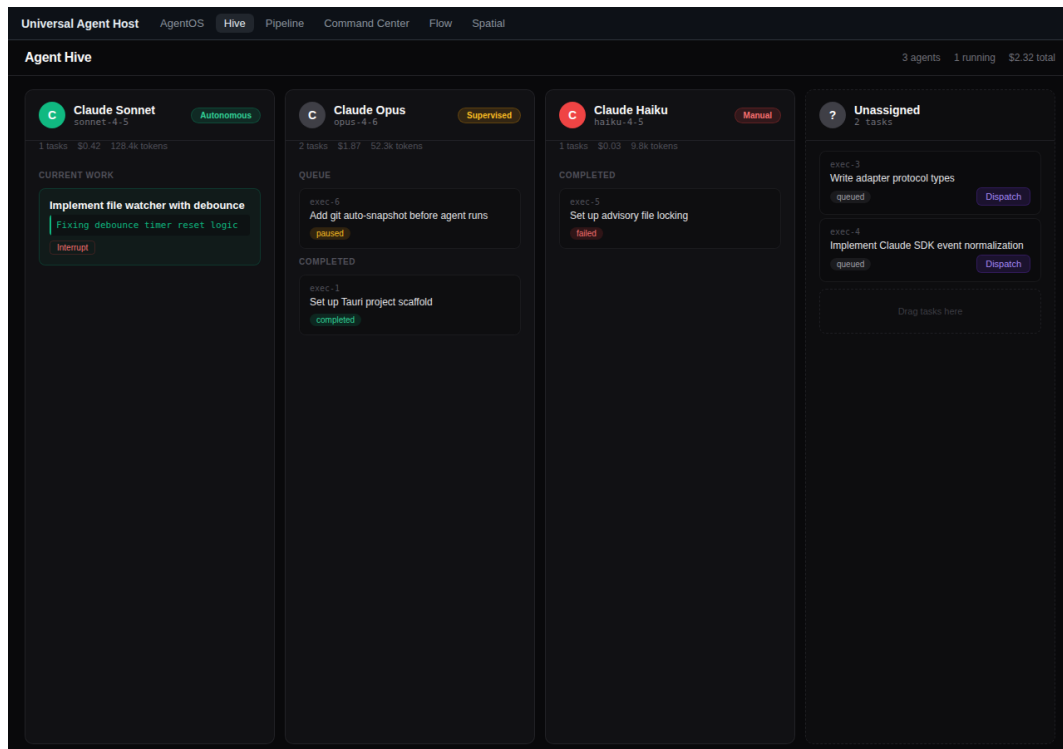
Magic Input

2. Hive

Agent-Centric Kanban with Trust Level Management

Layout

Horizontal agent columns with trust-level cycling



Full Page

Description

Hive organizes work by agent rather than by stage. Each agent gets a dedicated column showing their active and queued executions, with real-time status indicators and token usage tracking. The trust-level badge system lets operators cycle agents between Autonomous, Supervised, and Manual modes with a single click.

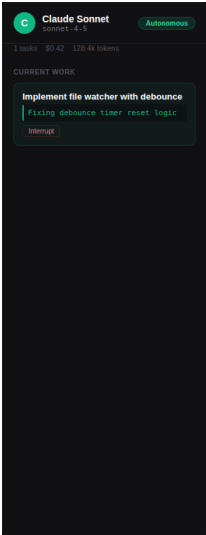
Design Reasoning

When managing multiple AI agents, the key question is often "what is each agent doing?" rather than "what stage is each task in?" Hive answers this by making agents the primary axis. The trust-level system addresses a core concern in agentic workflows: controlling how much autonomy each agent has.

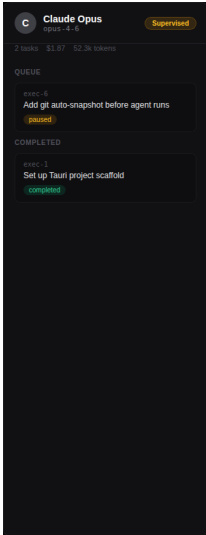
Key Components

- Per-agent columns with status badges
- Trust-level cycling (Autonomous → Supervised → Manual)
- Token usage tracking per agent
- Execution cards with status indicators
- Unassigned task overflow column

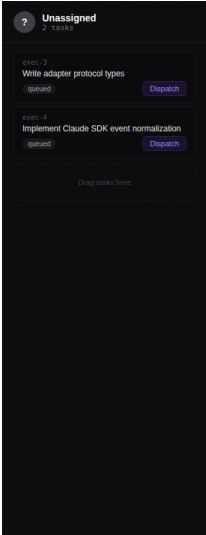
Hive — Component Details



Running Agent Column



Idle Agent Column



Unassigned Column



Trust Badge — Autonomous



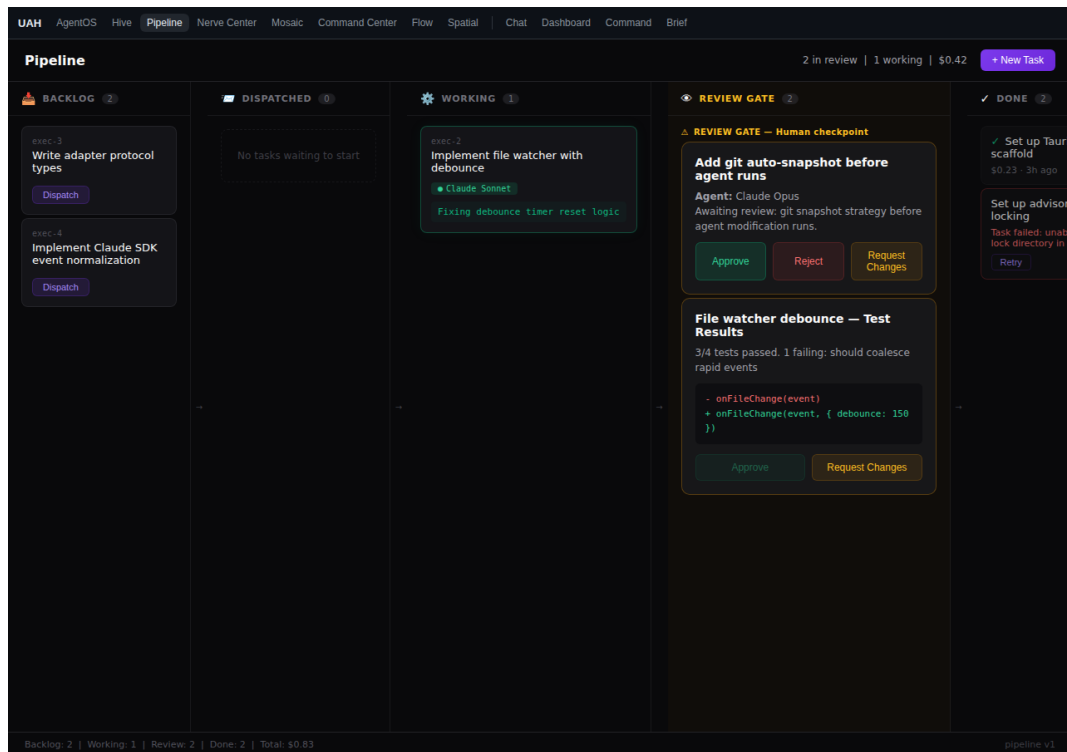
Trust Badge — Supervised

3. Pipeline

Stage-Gate Workflow with Human Review Checkpoints

Layout

Horizontal pipeline with 5 sequential stages connected by arrows



Full Page

Description

Pipeline models work as a sequential flow: Backlog → Dispatched → Working → Review Gate → Done. The review gate is a dedicated human checkpoint where tasks pause for approval, rejection, or change requests. Each stage shows item counts, and the footer provides aggregate statistics including cost tracking.

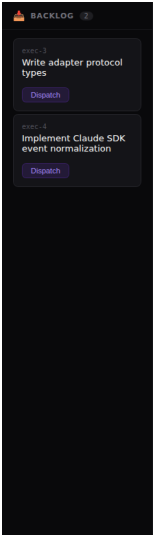
Design Reasoning

Many organizations require human-in-the-loop checkpoints before AI-generated work can be merged or deployed. Pipeline makes this review gate a first-class concept rather than an afterthought. The linear flow is intuitive for teams transitioning from traditional CI/CD pipeline thinking.

Key Components

- Five sequential pipeline stages with connector arrows
- Review gate with approve/reject/request-changes actions
- Diff preview for code changes under review
- Cost tracking per execution and aggregate
- Dispatch and retry actions

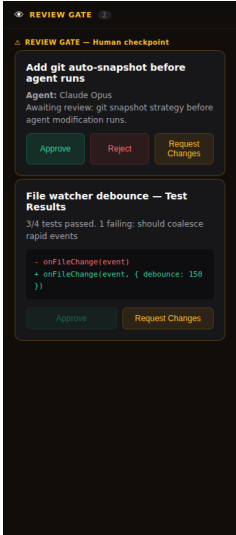
Pipeline — Component Details



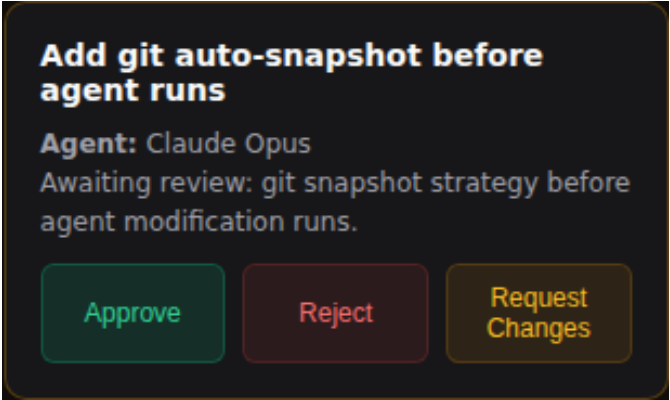
Backlog Stage



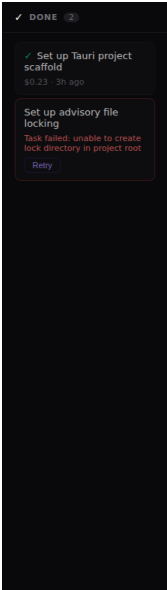
Working Stage



Review Gate



Review Card Detail



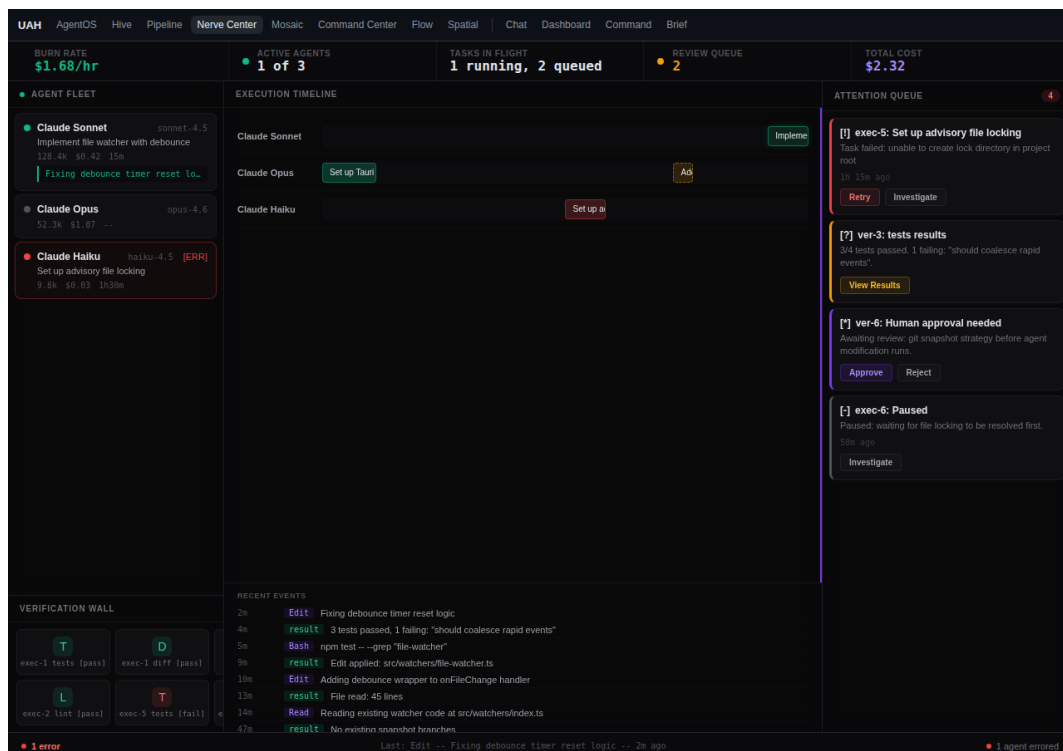
Done Stage

4. Nerve Center

Real-Time Monitoring & Intervention Dashboard

Layout

Multi-panel monitoring dashboard with metrics bar, fleet view, and alert system



Full Page

Description

Nerve Center is built for observation and rapid intervention. A metrics bar provides at-a-glance KPIs, the agent fleet panel shows real-time status of all agents, and the verification wall displays all quality gates across tasks. The attention queue surfaces items requiring human action, while the alert bar provides system-level notifications.

Design Reasoning

As the number of concurrent agents grows, operators need a "mission control" view that surfaces anomalies and intervention points. Nerve Center prioritizes monitoring over interaction, making it ideal for oversight roles or when managing large fleets of agents across multiple projects.

Key Components

- Real-time metrics bar with KPI cards
- Agent fleet status panel
- Verification wall showing all quality gates
- Execution timeline with temporal view
- Attention queue for items needing action
- System alert bar

Nerve Center — Component Details

Overall Cost
\$1.68/hr

Active Agents
1 of 3

Tasks in Flight
1 running, 2 queued

Agent Queue
2

Total Cost
\$2.32

Metrics Bar

AGENT FLEET

Claude Sonnet

sonnet-4.5

Implement file watcher with debounce

128.4k \$0.42 15m

Fixing debounce timer reset lo...

Claude Opus

opus-4.6

52.3k \$1.87 --

Claude Haiku

haiku-4.5 [ERR]

Set up advisory file locking

9.8k \$0.03 1h30m

Agent Fleet

VERIFICATION WALL

T

exec-1 tests [pass]

D

exec-1 diff [pass]

L

exec-2 lint [pass]

T

exec-5 tests [fail]

Verification Wall

EXECUTION TIMELINE

Claude Sonnet

Implement

Claude Opus

Set up Task

Ad

Claude Haiku

Set up a

RECENT EVENTS

2h

Edit

Fixing debounce timer reset logic

4h

result

3 tests passed, 1 failing "should coalesce rapid events"

5h

bash

python test --help file-watcher

6h

result

Edit endpoint src/watchers/file-watcher.ts

10h

Edit

Adding debounce wrapper to onFileChange handler

13h

result

File read: 45 lines

14h

Read

Reading existing watcher code at src/watchers/index.ts

15h

result

No previous watcher found

Execution Timeline

ATTENTION QUEUE

[!] exec-5: Set up advisory file locking

Task failed: unable to create lock directory in project root

1h 35m ago

Retry Investigate

[?] ver-3: tests results

3/4 tests passed, 1 failing "should coalesce rapid events"

View Results

[?] ver-6: Human approval needed

Awaiting review of proposed strategy before agent modification runs.

Approve Reject

[?] exec-6: Paused

Paused: waiting for file locking to be resolved first.

5m ago

Investigate

Attention Queue

1 error

1h 35m ago

Fixing debounce timer reset logic

1h 35m ago

1 approved

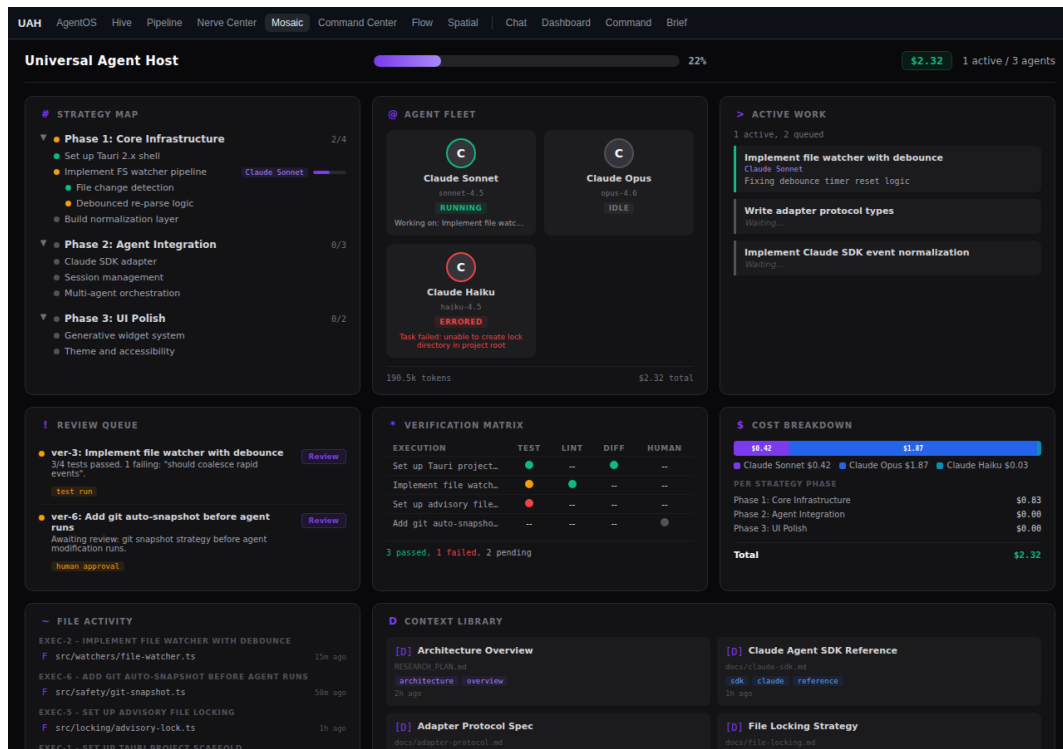
Alert Bar

5. Mosaic

Strategic Overview Radiator with 8 Information Cards

Layout

Responsive card grid (2×4 or 3-column) with dense information tiles



Full Page

Description

Mosaic presents eight information cards in a dense grid: Strategy progress, Agent fleet, Active work, Review queue, Verification matrix, Cost breakdown, File activity, and Context library. Each card is a self-contained widget providing a specific analytical lens on the system state.

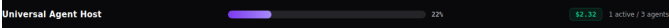
Design Reasoning

Information radiators work well in team settings where a dashboard is displayed on a shared screen. Mosaic's card-based approach means each team member can quickly scan the area they care about. The 8-card layout covers the full spectrum from strategy to cost to code changes.

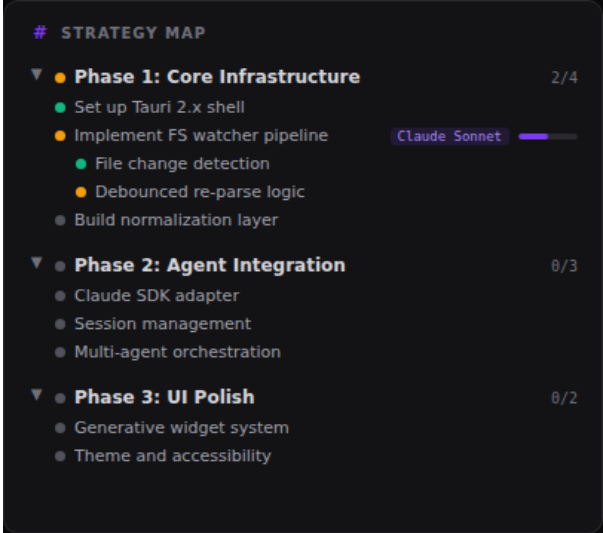
Key Components

- Strategy progress card with completion tracking
- Agent fleet card with status badges
- Active work card with running executions
- Review queue with pending items
- Verification matrix (pass/fail grid)
- Cost breakdown with per-execution costs
- File activity showing recent changes
- Context library with knowledge artifacts

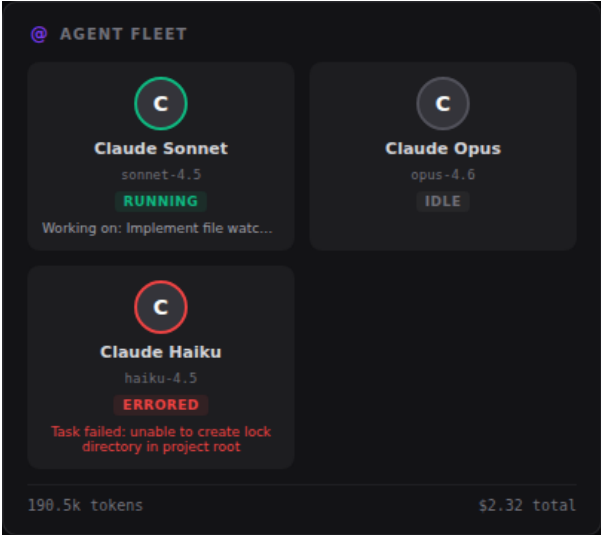
Mosaic — Component Details



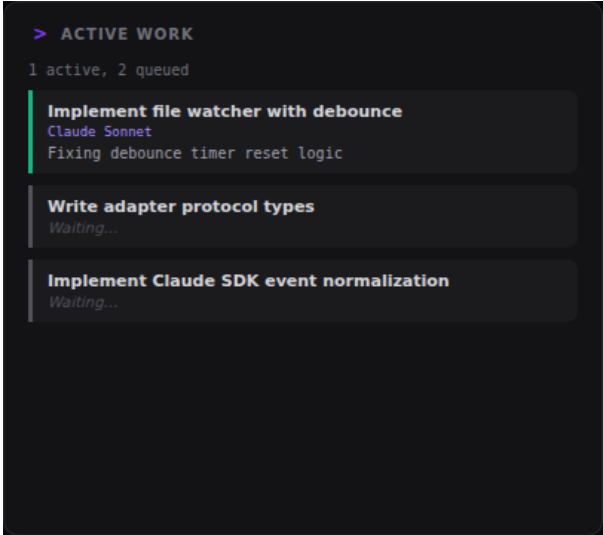
Header



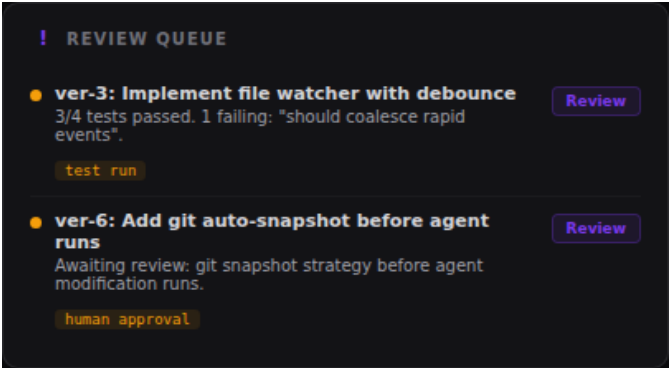
Strategy Card



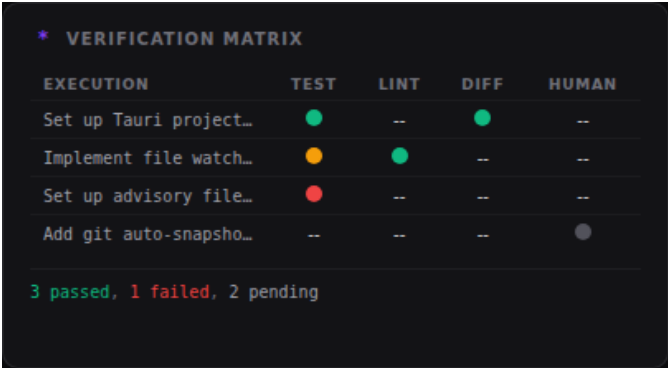
Agent Fleet Card



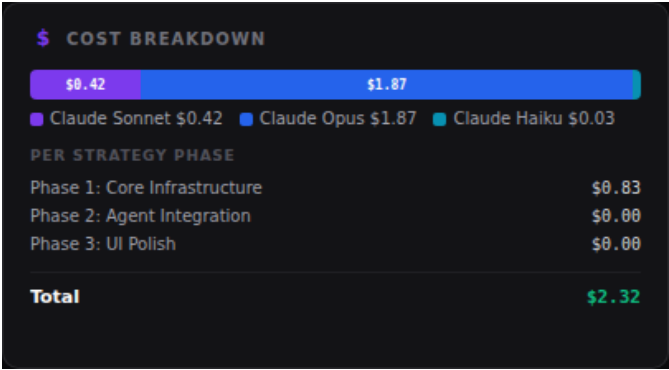
Active Work Card



Review Queue



Verification Matrix



Cost Breakdown

~ FILE ACTIVITY

EXEC-2 - IMPLEMENT FILE WATCHER WITH DEBOUNCE	
F src/watchers/file-watcher.ts	15m ago
EXEC-6 - ADD GIT AUTO-SNAPSHOT BEFORE AGENT RUNS	
F src/safety/git-snapshot.ts	50m ago
EXEC-5 - SET UP ADVISORY FILE LOCKING	
F src/locking/advisory-lock.ts	1h ago
EXEC-1 - SET UP TAURI PROJECT SCAFFOLD	
F src-tauri/Cargo.toml	2h ago
F src-tauri/src/main.rs	2h ago
F tauri.conf.json	2h ago

File Activity

D CONTEXT LIBRARY

[D] Architecture Overview RESEARCH_PLAN.md architecture overview 2h ago	[D] Claude Agent SDK Reference docs/claude-sdk.md sdk claude reference 1h ago
[D] Adapter Protocol Spec docs/adapter-protocol.md protocol adapter spec 45m ago	[D] File Locking Strategy docs/file-locking.md concurrency locking 30m ago

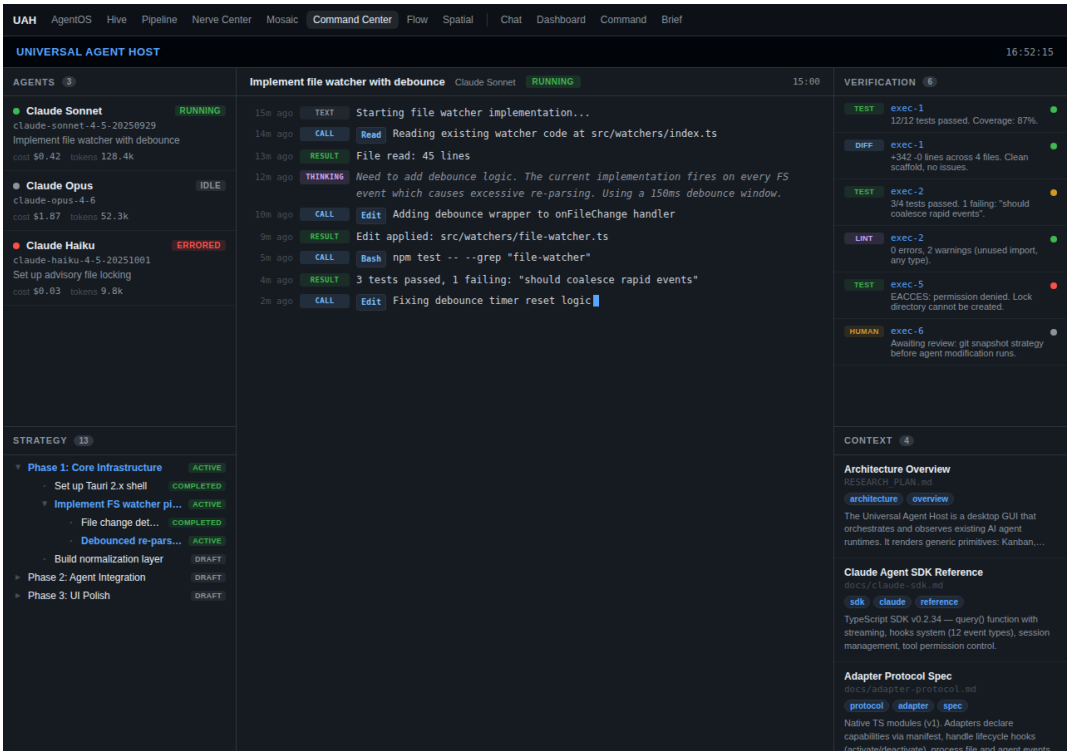
Context Library

6. Command Center

Unified 5-Panel Operational Command Interface

Layout

Header bar + symmetric 5-panel grid



Full Page

Description

Command Center divides the screen into five equal-weight panels: Agent Panel, Strategy Tree, Execution Log, Verification Panel, and Context Panel. A header bar with a system clock emphasizes real-time operation. Each panel provides deep access to one aspect of the system.

Design Reasoning

The symmetric grid ensures no single data type dominates, making Command Center ideal for operators who need equal access to all system facets. The real-time clock and operational aesthetics make it suited for environments where continuous monitoring is the norm.

Key Components

- Header with system clock
- Agent panel with status and capabilities
- Strategy tree with hierarchical decomposition
- Execution log with real-time streaming
- Verification panel with quality gate status
- Context panel with knowledge artifacts

Command Center — Component Details

AGENTS3

Claude Sonnet

claude-sonnet-4-5-20250929

Implement file watcher with debounce

cost \$0.42 tokens 128.4k

Claude Opus

claude-opus-4-6

cost \$1.87 tokens 52.3k

Claude Haiku

claude-haiku-4-5-20251001

Set up advisory file locking

cost \$0.03 tokens 9.8k

Agent Panel

STRATEGY13

Phase 1: Core Infrastructure

ACTIVE

Set up Tauri 2.x shell

COMPLETED

Implement FS watcher pl...

ACTIVE

File change det...

COMPLETED

Debounced re-pars...

ACTIVE

Build normalization layer

DRAFT

Phase 2: Agent Integration

DRAFT

Phase 3: UI Polish

DRAFT

Strategy Tree

Implement file watcher with debounceClaude SonnetRUNNING15:09

15h ago

TEST

Starting file watcher implementation...

15h ago

CALL

Read

Reading existing watcher code at src/watchers/index.ts

15h ago

RESULT

File read: 45 lines

15h ago

WARNING

Need to add debounce logic. The current implementation fires on every FS event which causes excessive re-parsing. Using a 100ms debounce window.

15h ago

CALL

Edit

Adding debounce wrapper to onFileChange handler

15h ago

RESULT

Edit applied: src/watchers/file-watcher.ts

15h ago

CALL

Run

npm test -- --grep "file-watcher"

15h ago

RESULT

3 tests passed, 1 failing: "should coalesce rapid events"

15h ago

CALL

Edit

Fixing debounce timer reset logic

Execution Log

VERIFICATION6

TEST

exec-1

12/12 tests passed. Coverage: 87%.

DIFF

exec-1

+342 -0 lines across 4 files. Clean scaffold, no issues.

TEST

exec-2

3/4 tests passed. 1 failing: "should coalesce rapid events".

LINT

exec-2

0 errors, 2 warnings (unused import, any type).

TEST

exec-5

EACCES: permission denied. Lock directory cannot be created.

HUMAN

exec-6

Awaiting review: git snapshot strategy before agent modification runs.

Verification Panel

CONTEXT4

Architecture Overview

RESEARCH_PLAN.md

architecture

overview

The Universal Agent Host is a desktop GUI that orchestrates and observes existing AI agent runtimes. It renders generic primitives: Kanban,...

Claude Agent SDK Reference

docs/claude-sdk.md

sdk

claude

reference

TypeScript SDK v0.2.34 — query() function with streaming, hooks system (12 event types), session management, tool permission control.

Adapter Protocol Spec

docs/adapter-protocol.md

protocol

adapter

spec

Native TS modules (v1). Adapters declare capabilities via manifest, handle lifecycle hooks (activate/deactivate), process file and agent events.

File Locking Strategy

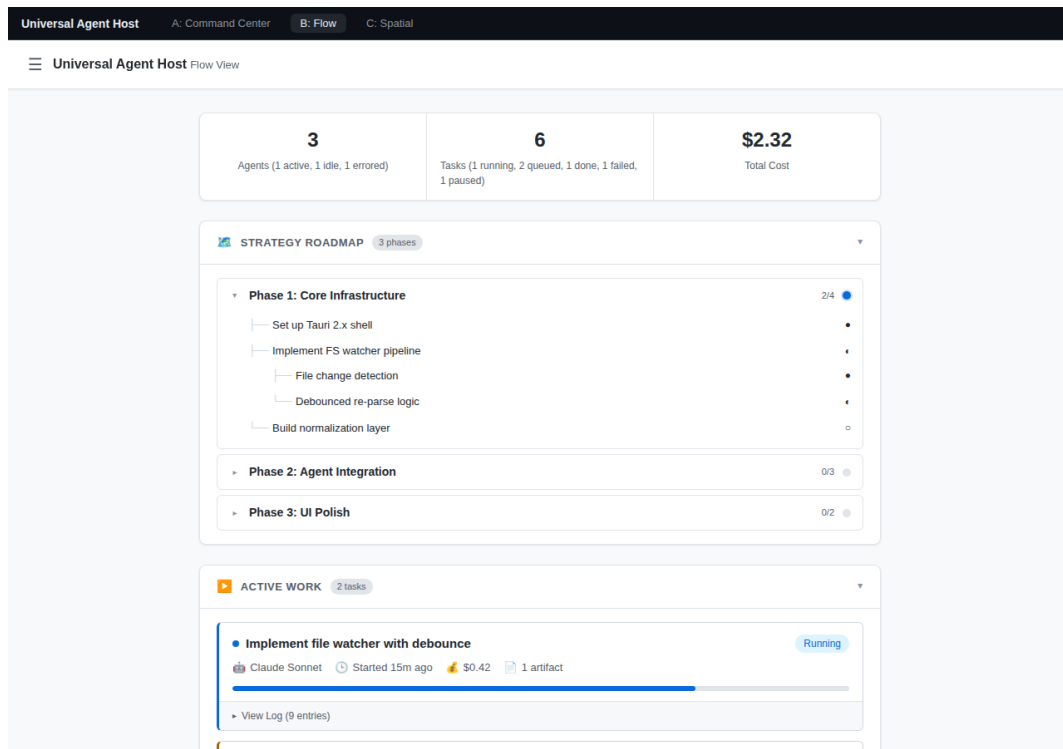
Context Panel

7. Flow

Narrative Vertical Scroll with Sequential Information Flow

Layout

Single-column vertical scroll with stacked sections



Full Page

Description

Flow presents information as a narrative journey: Summary Ribbon at the top, followed by Strategy Roadmap, Active Work, Recent Activity, Knowledge Base, and Verification Status. The linear layout encourages top-to-bottom reading and works well on both desktop and narrower viewports.

Design Reasoning

Not all users want a dense multi-panel dashboard. Flow caters to those who prefer a reading-oriented experience where context builds progressively. It's particularly effective for status updates and async reviews where someone needs to quickly catch up on what happened.

Key Components

- Summary ribbon with key metrics
- Strategy roadmap with progress bars
- Active work section with running tasks
- Recent activity feed
- Knowledge base with context artifacts
- Verification status overview

Flow — Component Details

3 Agents (1 active, 1 idle, 1 errored)	6 Tasks (1 running, 2 queued, 1 done, 1 failed, 1 paused)	\$2.32 Total Cost
--	---	---

Summary Ribbon

STRATEGY ROADMAP 3 phases	
Phase 1: Core Infrastructure2/4	Set up Tauri 2.x shell Implement FS watcher pipeline File change detection Debounce re-parse logic Build normalization layer
Phase 2: Agent Integration0/3	
Phase 3: UI Polish0/2	

Strategy Roadmap

ACTIVE WORK 2 tasks	
Implement file watcher with debounceRunning Claude SonnetStarted 15m ago\$0.421 artifact View Log (9 entries) Add git auto-snapshot before agent runsPaused Claude OpusStarted 50m ago\$0.151 artifact View Log (4 entries)	

Active Work

RECENT ACTIVITY 4 items	
Set up Tauri project scaffold Claude Opuscompleted2h ago 3 artifacts	\$0.23
Write adapter protocol types Unassignedqueuedqueued	
Implement Claude SDK event normalization Unassignedqueuedqueued	
Set up advisory file locking Claude Haikufailed1h ago 1 artifact	\$0.03

Recent Activity

KNOWLEDGE BASE 4 items	
Architecture Overview The Universal Agent Host is a desktop GUI that orchestrates and observes existing AI agent runtimes. It renders generic primitives: Kanban, Tree, Wiki views. architectureoverview RESEARCH_PLAN.md2h ago	
Claude Agent SDK Reference TypeScript SDK v0.2.34 — query() function with streaming, hooks system (12 event types), session management, tool permission control. sdkclaudereference docs/claude-sdk.md1h ago	
Adapter Protocol Spec Native TS modules (v1). Adapters declare capabilities via manifest, handle lifecycle hooks (activate/deactivate), process file and agent events. protocoladapterspec docs/adapter-protocol.md45m ago	
File Locking Strategy Advisory file-level locking using .agent-lock/ directory. Agents check lock before writing. FS watcher + debounced re-parse for user-editable files. concurrencylocking docs/file-locking.md30m ago	

Knowledge Base

VERIFICATION STATUS 6 checks	
Set up Tauri project scaffoldexec-1	
Test Run	Passed12/12 tests passed. Coverage: 87%.test-results/exec-1.json
Diff Review	Passed+342 -0 lines across 4 files. Clean scaffold, no issues.
Implement file watcher with debounceexec-2	
Test Run	Needs Review3/4 tests passed. 1 failing: "should coalesce rapid events".test-results/exec-2.json
Lint	Passed0 errors, 2 warnings (unused import, any type).
Set up advisory file lockingexec-5	
Test Run	FailedEACCES: permission denied. Lock directory cannot be created.test-results/exec-5.json
Add git auto-snapshot before agent runsexec-6	
Human Approval	PendingAwaiting review: git snapshot strategy before agent modification runs.

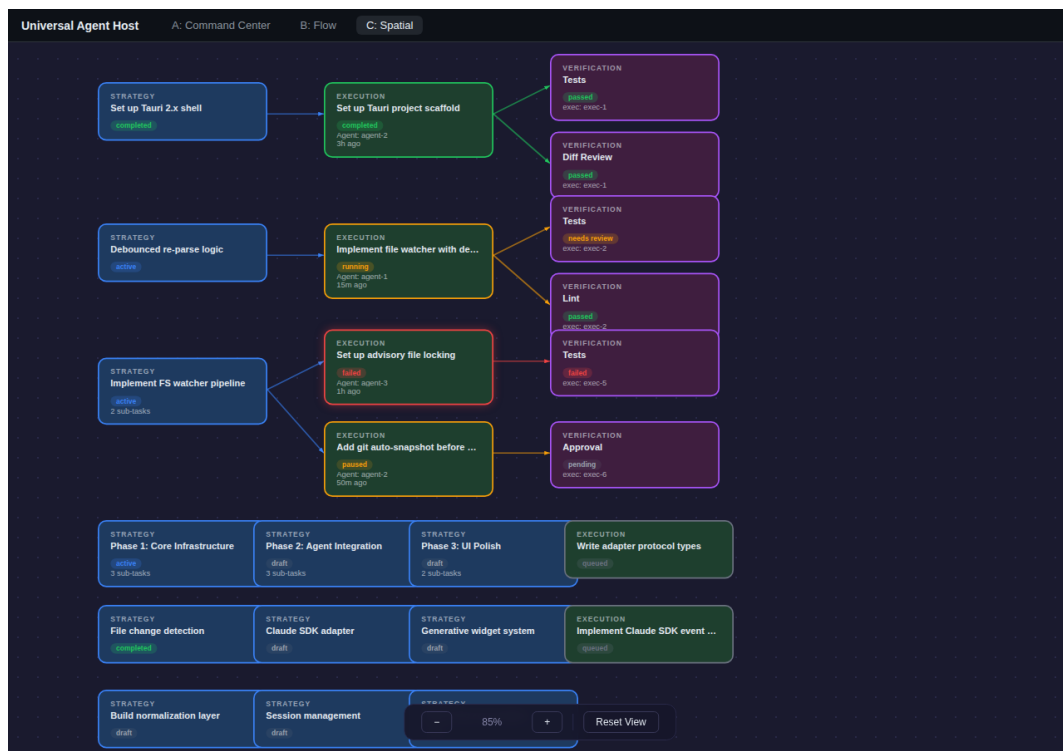
Verification Status

8. Spatial

Graph-Based Canvas for Entity Relationship Mapping

Layout

Free-form 2D canvas with interactive nodes and edges



Full Page

Description

Spatial maps strategies, executions, verifications, contexts, and agents as nodes on a 2D canvas, connected by colored edges that represent relationships. Clicking a node opens a detail sidebar with full information. This approach reveals hidden relationships and dependencies that tabular views miss.

Design Reasoning

Complex agentic workflows involve many interconnected entities. Spatial visualization helps operators discover unexpected dependencies, bottlenecks, and clusters. The interactive canvas supports exploratory analysis that structured layouts cannot.

Key Components

- Interactive 2D canvas with positioned nodes
- Five node types: strategy, execution, verification, context, agent
- Color-coded edges showing relationships
- Click-to-select with detail sidebar
- Status-colored node indicators

Spatial — Component Details



Canvas Area



With Detail Sidebar

Phase 1: Core Infrastructure

- STATUS: Active
- SOURCE FILE: `PLAN.md`
- CHILDREN (3)
 - Set up Tauri 2.x shell: Completed
 - Implement FS watcher pipeline: Active
 - Build normalization layer: Idle
- METADATA: `priority: high`

Detail Sidebar Content

STARTUP VARIANTS

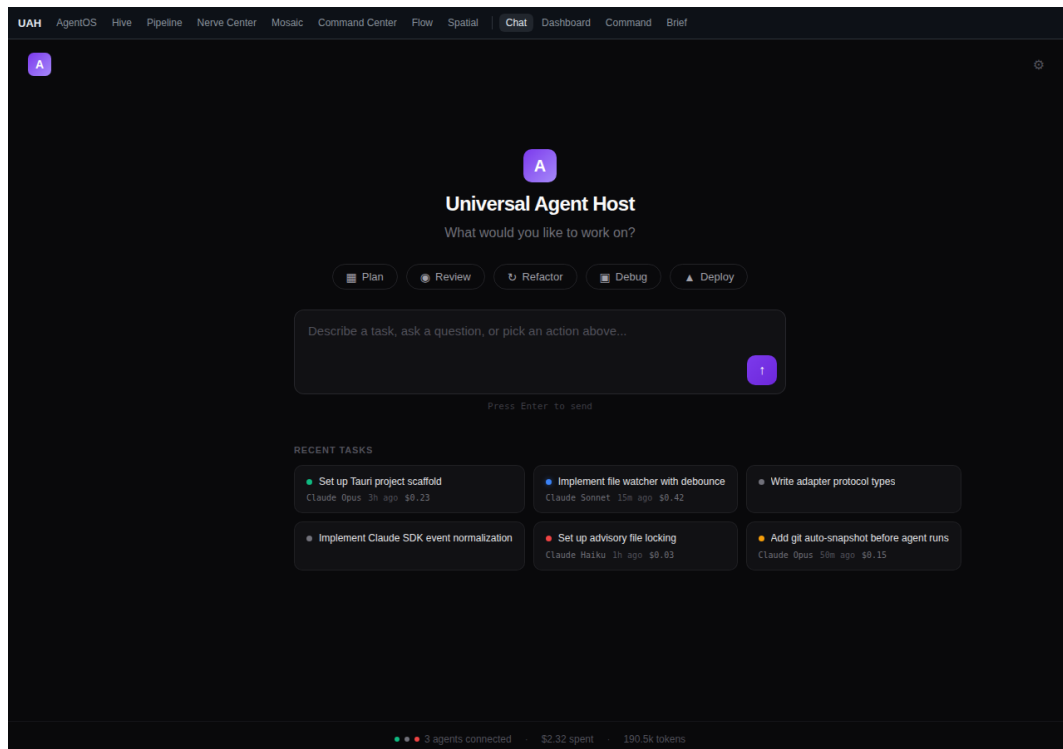
Four landing page variants exploring how users begin new sessions: through chat, dashboards, command palettes, or mission briefings.

9. Startup: Chat

Chat-First Interface with Quick Action Chips

Layout

Centered chat interface with action chips and recent work sidebar



Full Page

Description

The Chat startup presents a clean, centered input field with a greeting and quick-action chips (Plan, Review, Refactor, Debug, Deploy). Recent tasks are shown below for context. This pattern mirrors popular AI chat interfaces, making it immediately familiar.

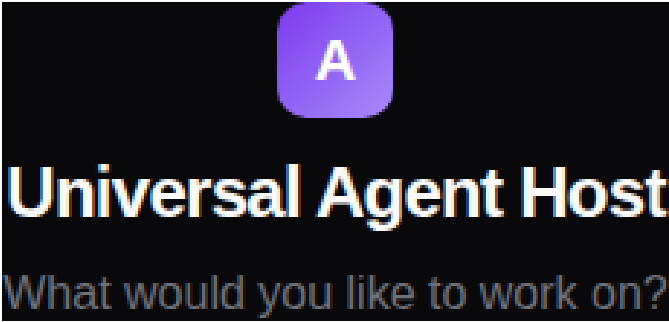
Design Reasoning

For users who think in terms of conversations rather than dashboards, a chat-first interface lowers the barrier to starting work. Action chips provide discoverability for common operations without requiring users to know command syntax.

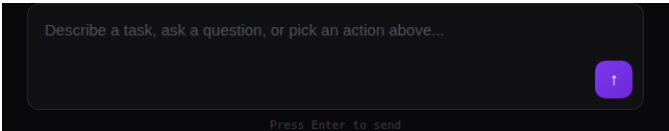
Key Components

- Centered hero area with greeting
- Quick action chips for common operations
- Chat input with rich text support
- Recent tasks list for context

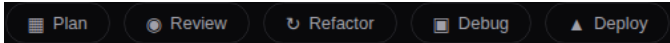
Startup: Chat — Component Details



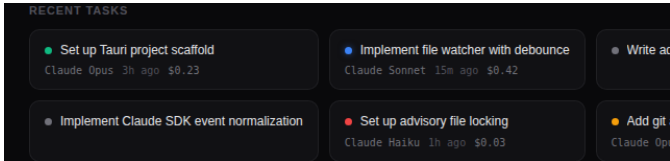
Hero Area



Chat Input



Action Chips



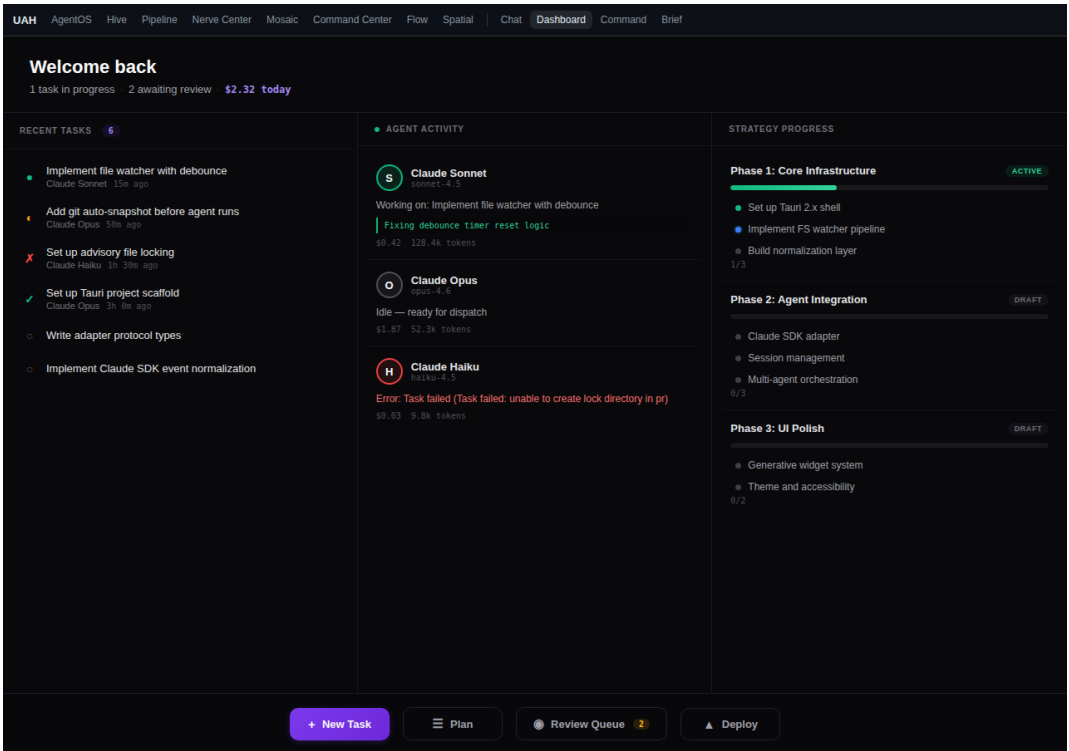
Recent Tasks

10. Startup: Dashboard

Status Overview with Quick Actions

Layout

Welcome header + card grid with tasks, agents, strategy, and actions



Full Page

Description

The Dashboard startup provides an at-a-glance overview of the current state: recent tasks, agent activity, strategy progress, and quick action buttons. It's designed as a "home base" that helps users decide what to do next based on the current system state.

Design Reasoning

Returning users need context before diving in. The dashboard answers "what happened while I was away?" and "what should I do next?" in a single glance. Quick actions reduce the clicks needed to start common workflows.

Key Components

- Welcome header with personalized greeting
- Recent tasks with status indicators
- Agent activity summary
- Strategy progress with completion bars
- Quick action buttons

Startup: Dashboard — Component Details

Welcome back

1 task in progress

2 awaiting review

\$2.32 today

Welcome Header

RECENT TASKS

Implement file watcher with debounce

Claude Sonnet · 15m ago

Add git auto-snapshot before agent runs

Claude Opus · 51m ago

Set up advisory file locking

Claude Haiku · 1h 20m ago

Set up Tauri project scaffold

Claude Opus · 3h 10m ago

Write adapter protocol types

Implement Claude SDK event normalization

Recent Tasks

AGENT ACTIVITY

S

Claude Sonnet

sonnet-4.5

Working on: Implement file watcher with debounce

Fixing debounce timer reset logic

58:42 · 128.4k tokens

O

Claude Opus

opus-4.6

Idle — ready for dispatch

11:67 · 52.3k tokens

H

Claude Haiku

haiku-4.5

Error: Task failed (Task failed: unable to create lock directory in pr)

58:63 · 9.8k tokens

Agent Activity

STRATEGY PROGRESS

Phase 1: Core Infrastructure

ACTIVE

Set up Tauri 2.x shell

Implement FS watcher pipeline

Build normalization layer

1/3

Phase 2: Agent Integration

DRAFT

Claude SDK adapter

Session management

Multi-agent orchestration

0/3

Phase 3: UI Polish

DRAFT

Generative widget system

Theme and accessibility

0/2

Strategy Progress

+ New Task

Plan

Review Queue 2

Deploy

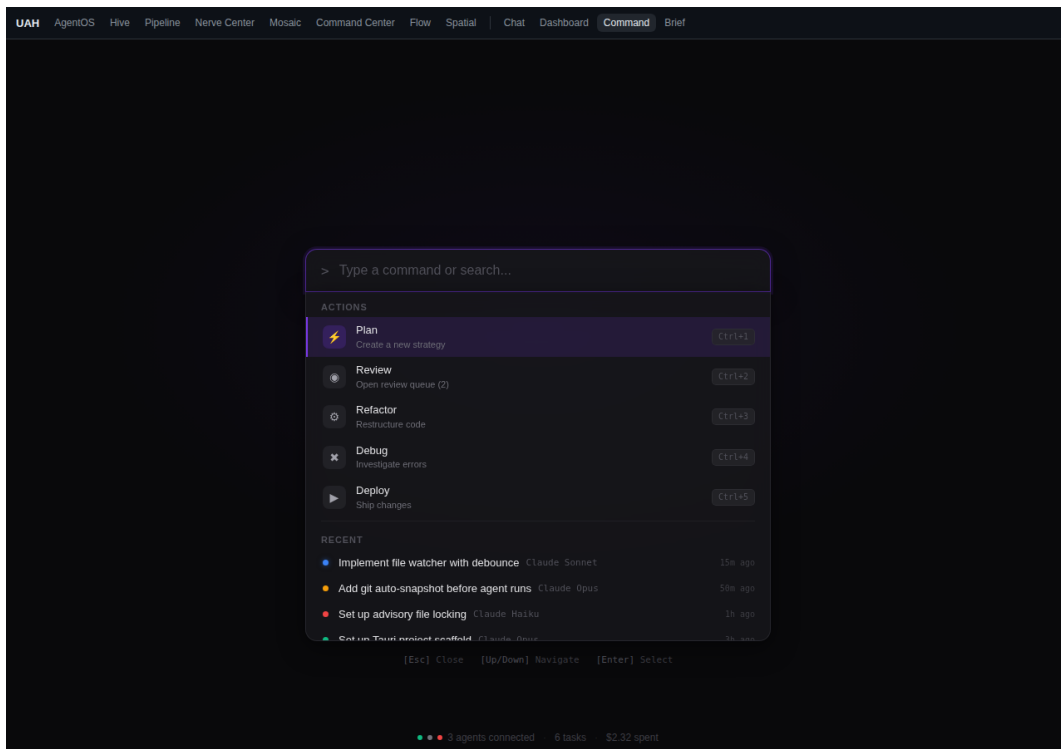
Quick Actions

11. Startup: Command

Raycast/Spotlight-Style Command Palette

Layout

Centered command input with results list and keyboard hints



Full Page

Description

The Command startup mimics Raycast or Spotlight: a focused search/command input with filterable results and keyboard shortcuts (Ctrl+1 through Ctrl+5). Recent items and available actions are shown below the input for quick access.

Design Reasoning

Power users prefer keyboard-driven workflows over mouse navigation. The command palette pattern is proven in tools like VS Code, Raycast, and Spotlight. It provides the fastest path from intent to action for experienced users.

Key Components

- Centered command input with search
- Filterable results panel
- Keyboard shortcuts for quick actions
- Keyboard hints footer

Startup: Command — Component Details

Type a command or search...

ACTIONS

 **Plan**
Create a new strategy

 **Review**
Open review queue (2)

 **Refactor**
Restructure code

 **Debug**
Investigate errors

 **Deploy**
Ship changes

RECENT

 Implement file watcher with debounce

Claude Sonnet

15m ago

 Add git auto-snapshot before agent runs

Claude Opus

50m ago

 Set up advisory file locking

Claude Haiku

1h ago

 Set up Turb project scaffold

Claude Opus

3h ago

Command Input

Results Panel

[Esc] Close [Up/Down] Navigate [Enter] Select

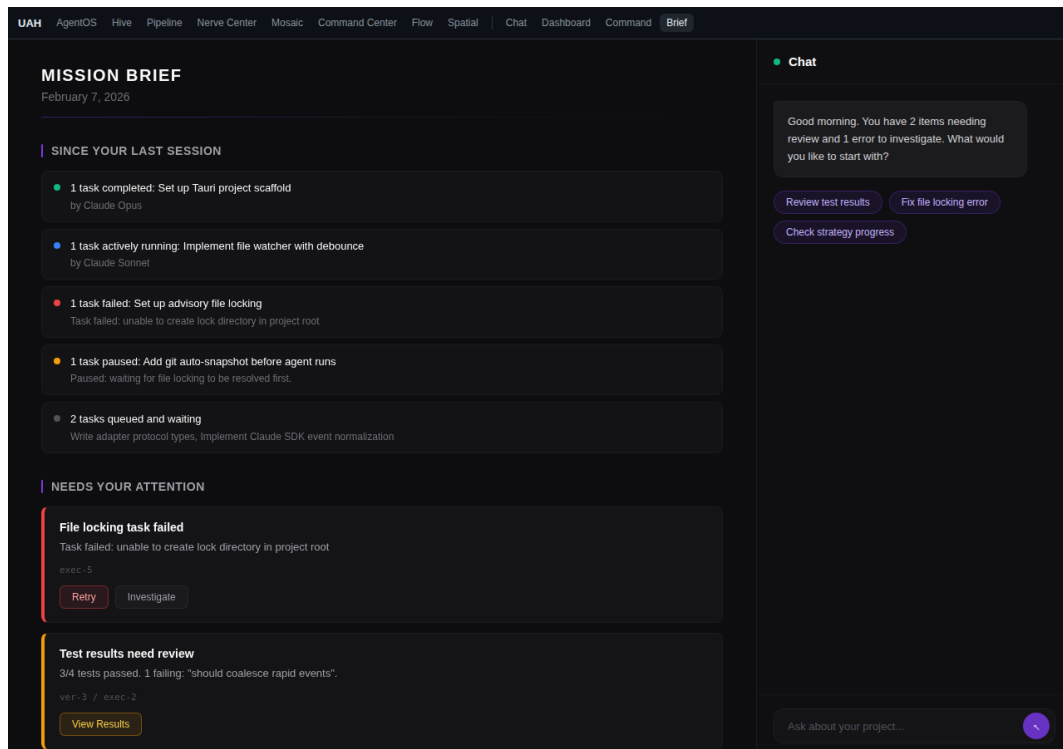
Keyboard Hints

12. Startup: Brief

Mission Briefing with Chat Sidebar

Layout

Two-column: briefing content (left) + chat panel (right)



Full Page

Description

The Brief startup presents a "mission briefing" with session summary, attention items, strategy progress, and quick actions on the left, with a chat sidebar on the right for immediate conversation. It combines the dashboard and chat approaches.

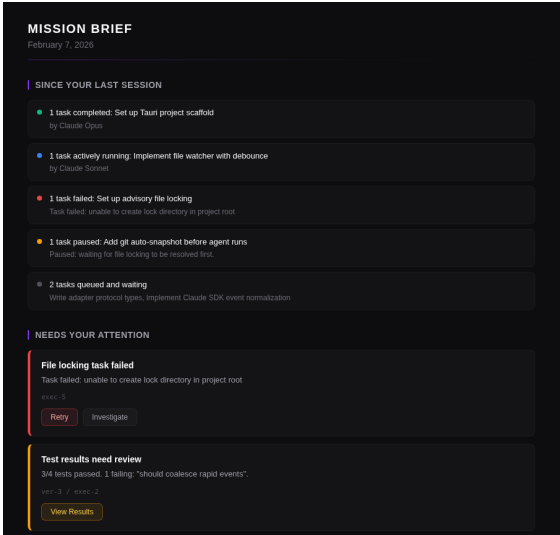
Design Reasoning

The mission briefing metaphor sets a tone of intentionality: users review the state of affairs before giving orders. The chat sidebar means they can immediately act on what they see without navigating away. This dual-panel approach serves both analytical and conversational work styles.

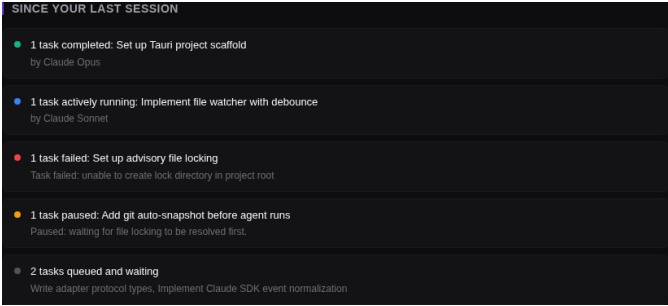
Key Components

- Session summary with key statistics
- Attention section with items requiring action
- Strategy progress overview
- Quick action buttons
- Chat sidebar for immediate interaction

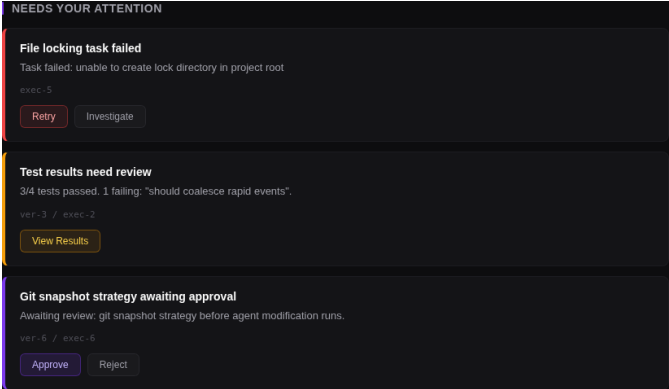
Startup: Brief — Component Details



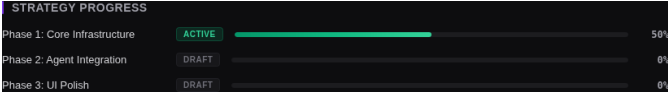
Brief Content



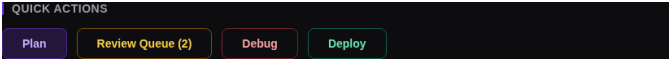
Session Summary



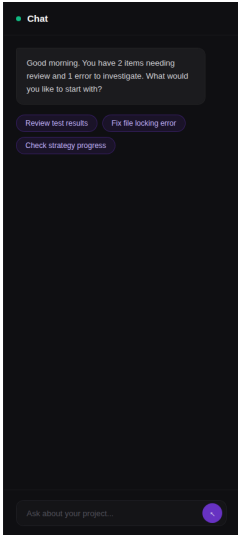
Attention Section



Strategy Progress



Quick Actions



Chat Panel

WORKFLOW VARIANTS

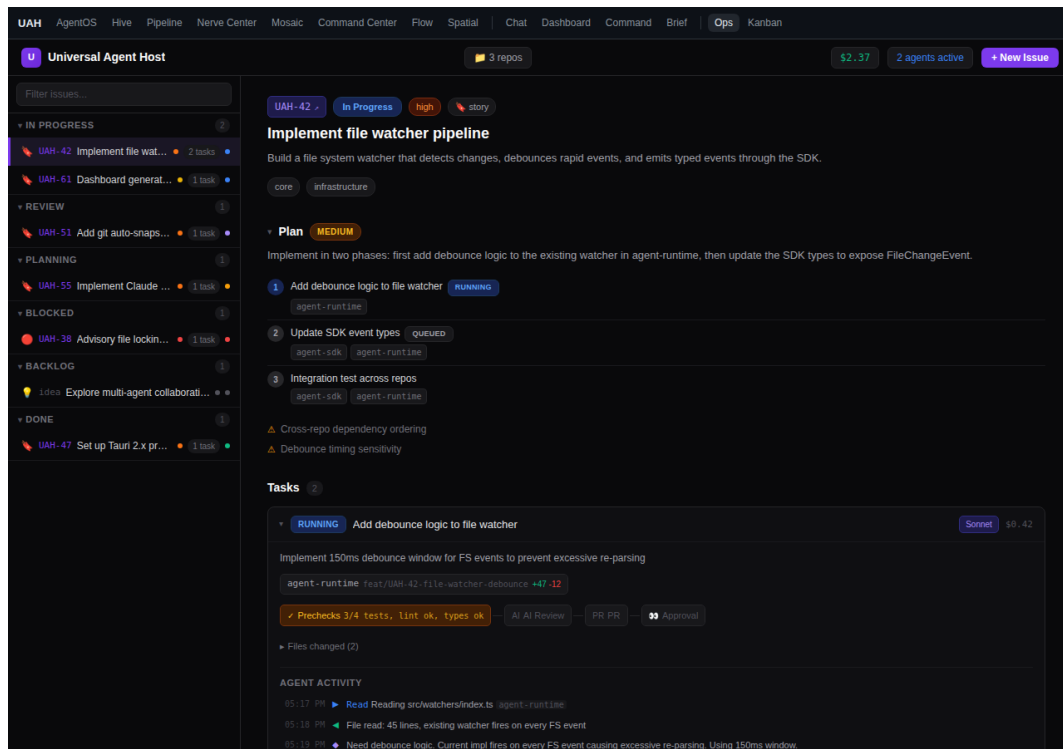
Two task managers built on a new workflow-oriented type system that models real development workflows: Projects → Issues → Plans → Tasks → 4-stage Verification Pipeline (Prechecks → AI Review → PR → Approval).

13. Ops

Focused Task Manager Bridging Dev Workflow

Layout

Two-panel: issue list (left) + detail view (right) with plan, tasks, and verification pipeline



Full Page

Description

Ops is a workflow-oriented task manager built on a new type system that models real development workflows: Projects contain Issues (linked to Jira/DevOps), Issues have Plans with steps, Plans produce Tasks assigned to agents, and each Task goes through a 4-stage verification pipeline (Prechecks → AI Review → PR → Approval). The left panel lists issues grouped by status with icons and priority indicators. The right panel shows full detail including the plan, tasks with agent assignments, and the verification pipeline visualization.

Design Reasoning

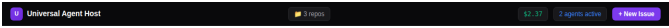
Previous variants used abstract primitives (Context, Strategy, Execution) that didn't map to how developers actually think about work. Ops bridges this gap by using familiar concepts: issues, plans, tasks, PRs, and reviews. The verification pipeline is the key innovation — bundling lint/types/tests/build into a single Prechecks gate, then adding AI Review, PR creation, and human Approval as distinct stages that each provide meaningful checkpoints.

Key Components

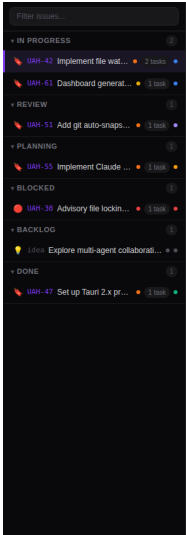
- Issue list grouped by status (in progress, review, blocked, etc.)
- Issue detail with external system links (Jira, GitHub)
- Plan view with step-by-step breakdown
- Task cards with agent assignments and repo tags
- 4-stage verification pipeline visualization
- Multi-repo project support

- Cost tracking and time estimation

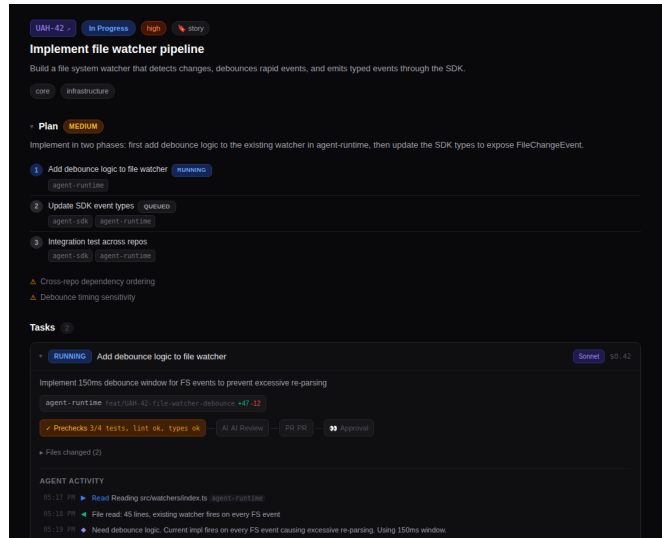
Ops — Component Details



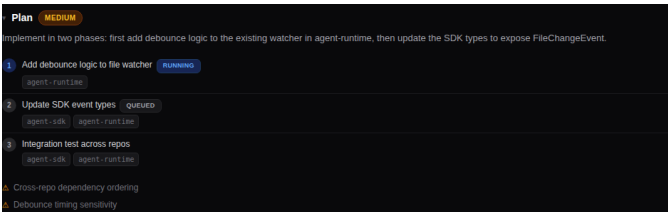
Top Bar



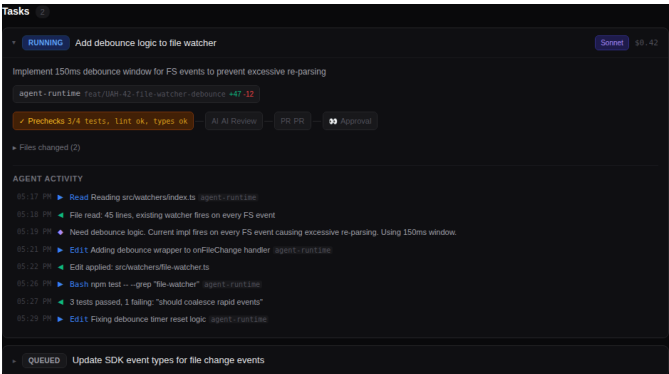
Issue List



Issue Detail



Plan Section



Tasks Section



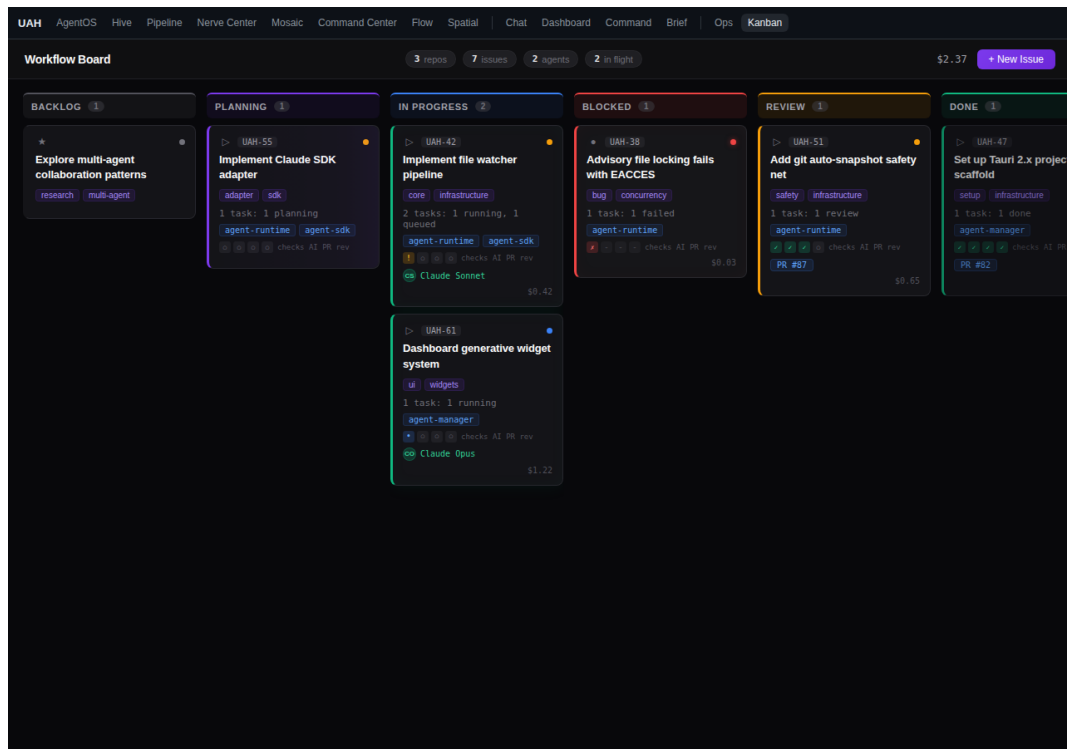
Verification Pipeline

14. Kanban

Workflow Board with Issue Lifecycle Columns

Layout

Six-column kanban board: Backlog → Planning → In Progress → Blocked → Review → Done



Full Page

Description

Kanban provides a visual board where issues flow through six lifecycle columns. Each card shows the issue title, priority, assigned agent, repo tags, and a compact verification pipeline status (dots for each stage). Cards can be expanded to reveal full detail including plan steps, task breakdowns, and PR information. The header shows aggregate statistics and cost tracking.

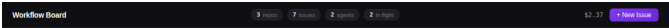
Design Reasoning

While Ops focuses on detail for a single issue, Kanban provides the big picture across all issues. The six columns map to the natural issue lifecycle in software development, making WIP limits and bottlenecks visible at a glance. The compact verification dots on each card let operators quickly spot which tasks are stuck in which verification stage without clicking through.

Key Components

- Six lifecycle columns with item counts
- Issue cards with priority, agent, and repo tags
- Compact verification pipeline dots per card
- Expandable card detail with full task/plan info
- Blocked column for visibility into impediments
- Aggregate stats in header

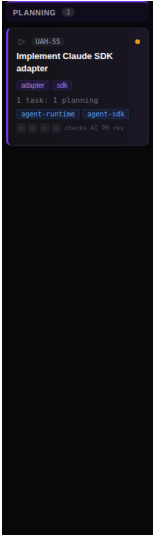
Kanban — Component Details



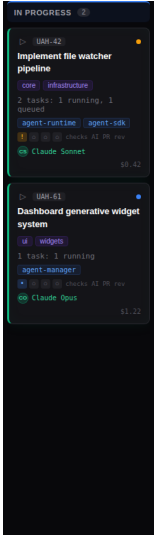
Header



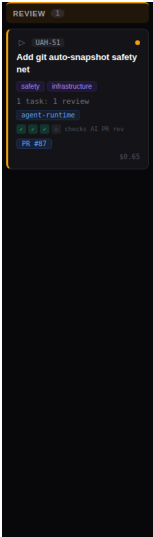
Backlog Column



Planning Column



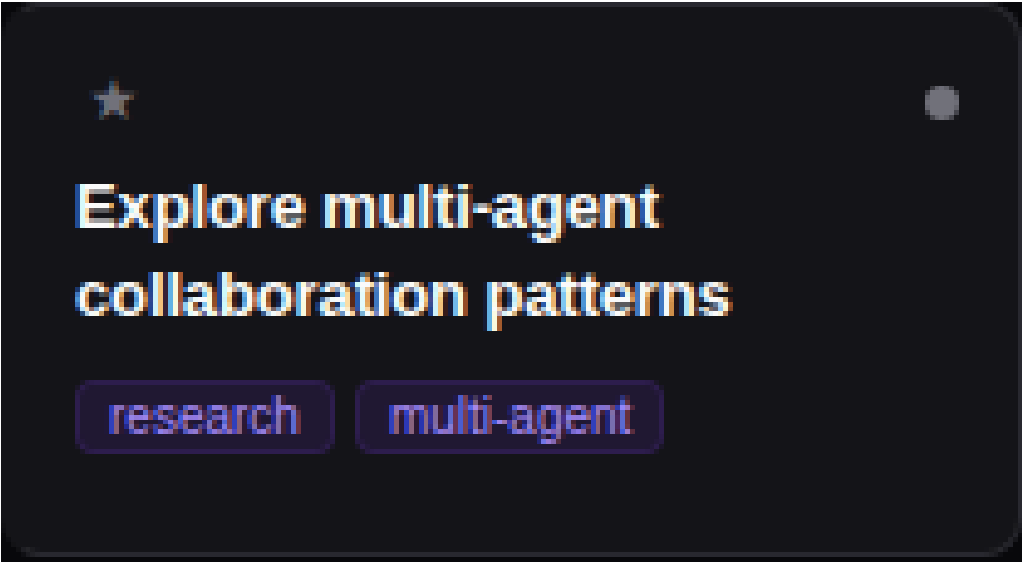
In Progress Column



Review Column



Done Column



Issue Card

Appendix A: Comparative Analysis

The table below compares all 14 variants across layout type, primary focus, target user, and underlying data model.

Variant	Layout	Focus	Best For	Data Model
AgentOS	Sidebar + Tabs	Deep work	Power users	Original
Hive	Agent columns	Agent trust	Multi-agent ops	Original
Pipeline	Horizontal stages	Quality gates	Review workflows	Original
Nerve Center	Dashboard panels	Monitoring	Fleet oversight	Original
Mosaic	Card grid	Overview	Team radiator	Original
Command Center	5-panel grid	Operations	Control rooms	Original
Flow	Vertical scroll	Narrative	Async reviews	Original
Spatial	2D canvas	Relationships	Dependency analysis	Original
Chat	Centered chat	Quick start	New sessions	Original
Dashboard	Card grid	Status recap	Returning users	Original
Command	Palette	Keyboard	Power users	Original
Brief	Dual panel	Briefing + chat	Decision makers	Original
Ops	List + detail	Issue workflow	Dev teams	Workflow
Kanban	6-col board	WIP visibility	Project managers	Workflow

Appendix B: Information Architecture Dimensions

Each variant makes different trade-offs along several architectural dimensions. Understanding these trade-offs helps choose the right variant for a given context.

Information Density

How much information is visible at once without scrolling.

Low: Flow, Chat, Command

Medium: Pipeline, Spatial, Brief, Kanban

High: AgentOS, Hive, Nerve Center, Mosaic, Command Center, Ops

Interaction Model

How users primarily interact with the interface.

Read-only / Monitor: Nerve Center, Mosaic, Flow, Dashboard

Select & Inspect: Spatial, Ops, Kanban, Brief

Direct Manipulation: AgentOS, Hive, Pipeline, Command Center, Chat, Command

Primary Axis

What the UI is organized around.

Agent-centric: Hive

Stage/Workflow: Pipeline, Kanban

Issue/Task: Ops, AgentOS

Strategy/Hierarchy: Mosaic, Command Center, Flow

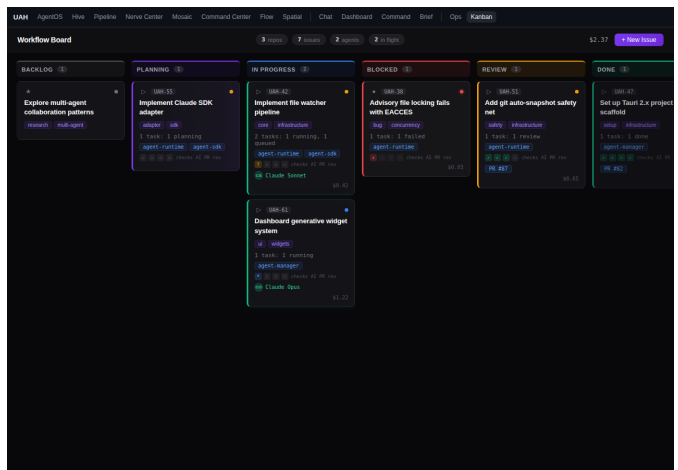
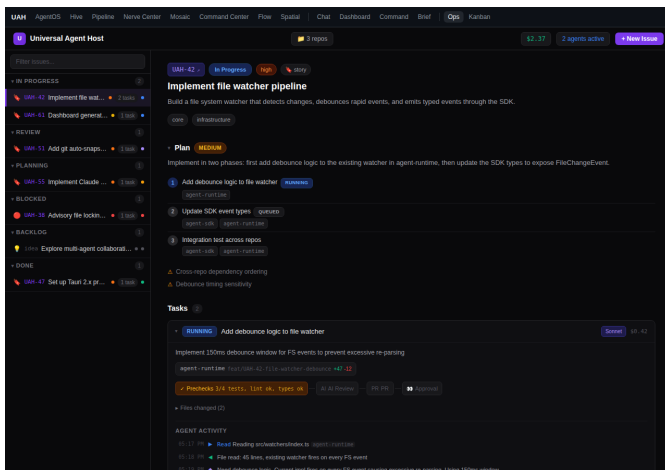
Entity Graph: Spatial

Action/Command: Chat, Command, Brief

Time/Events: Nerve Center, Dashboard

Side-by-Side: Ops vs Kanban

The two workflow variants represent complementary views of the same data. Ops provides depth for a single issue; Kanban provides breadth across all issues.

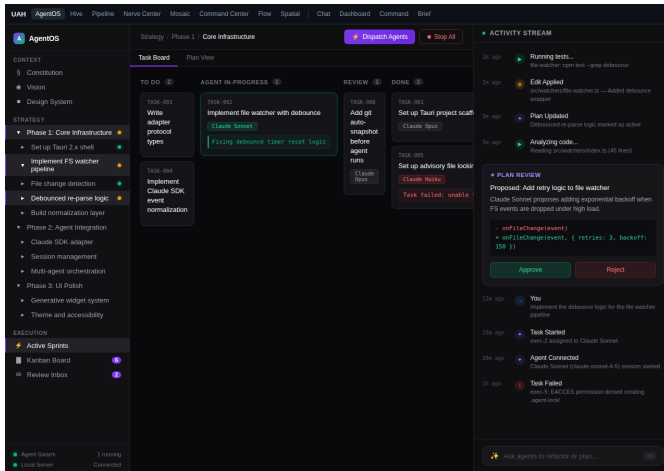


Ops — Detail-first, single issue focus

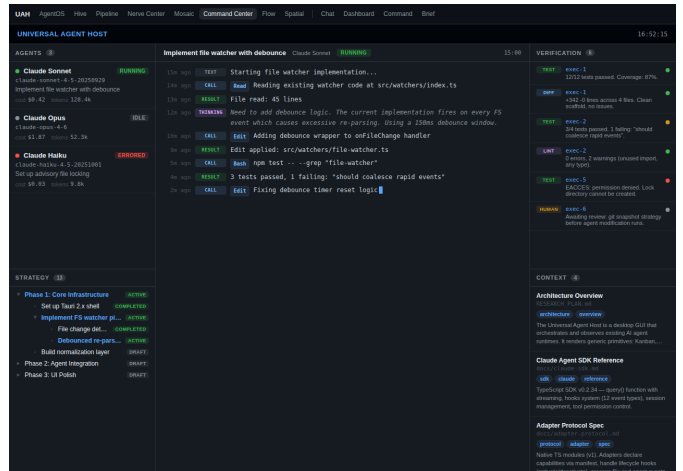
Kanban — Overview-first, all issues at once

Side-by-Side: AgentOS vs Command Center

Both are high-density multi-panel layouts, but AgentOS uses a sidebar hierarchy while Command Center uses a symmetric grid.



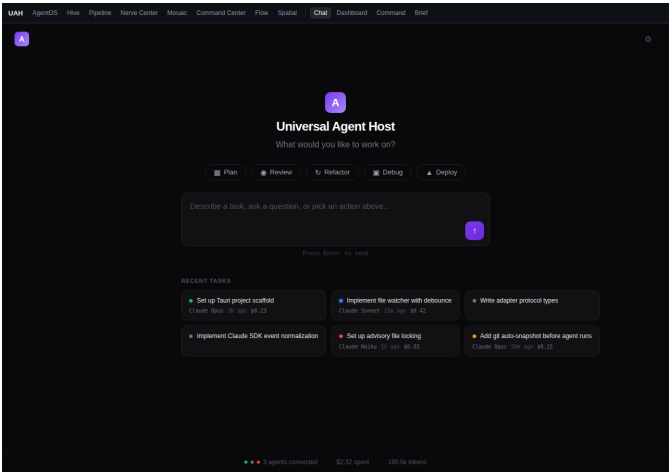
AgentOS — Sidebar + main + console



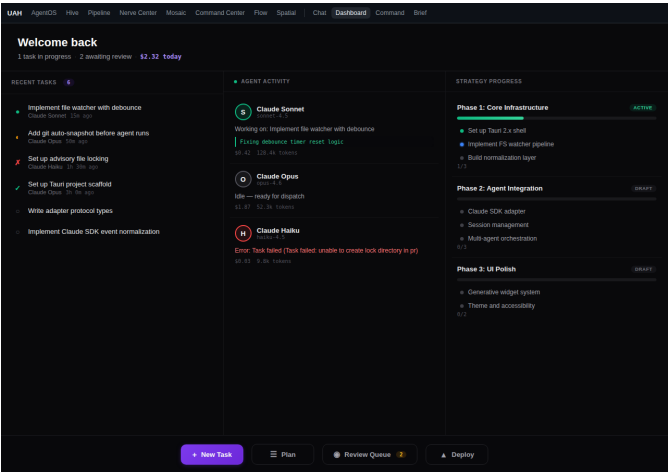
Command Center — Symmetric 5-panel grid

Side-by-Side: Startup Variants

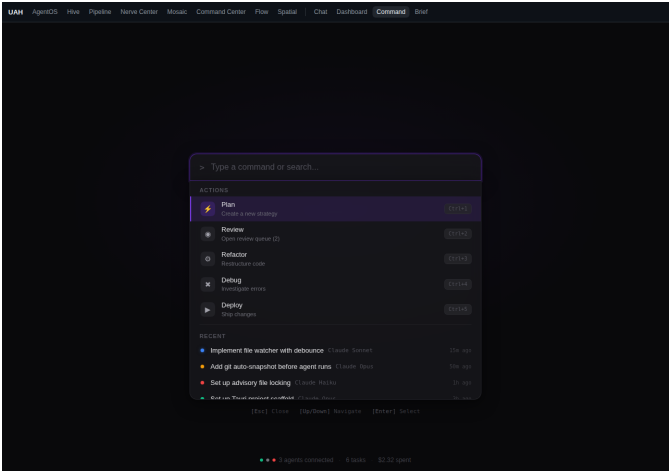
Four different approaches to the initial user experience, each optimized for a different interaction style.



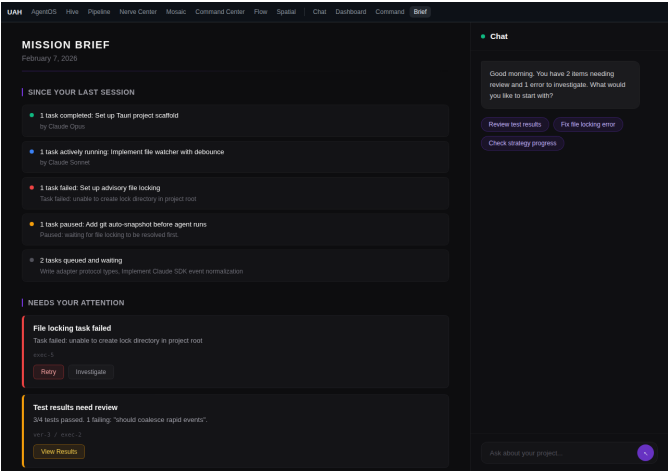
Chat — Conversation-first



Dashboard — Status overview



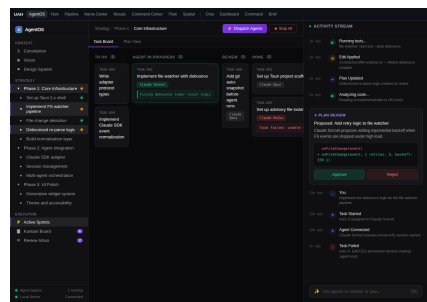
Command — Keyboard-driven



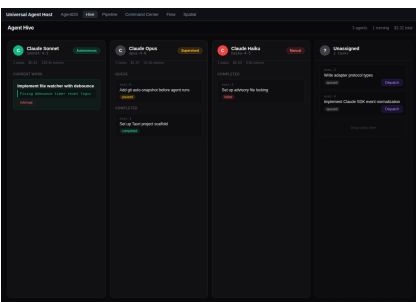
Brief — Briefing + chat dual-panel

All Variants at a Glance

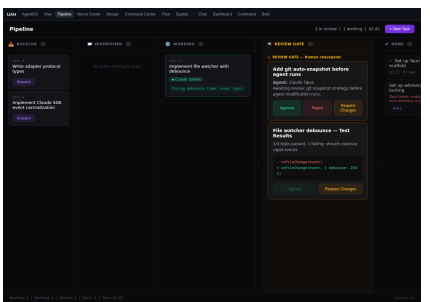
Thumbnail overview of every variant's full-page screenshot, showing the range of layout approaches explored.



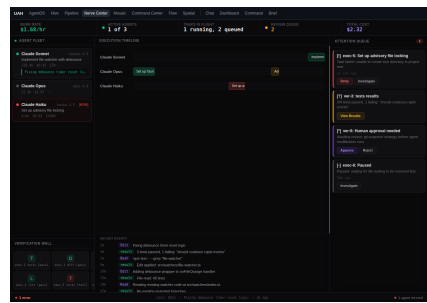
AgentOS



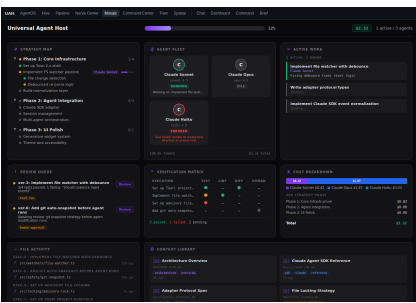
Hive



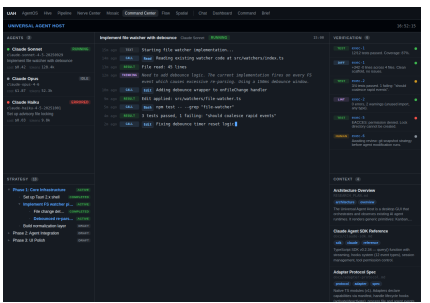
Pipeline



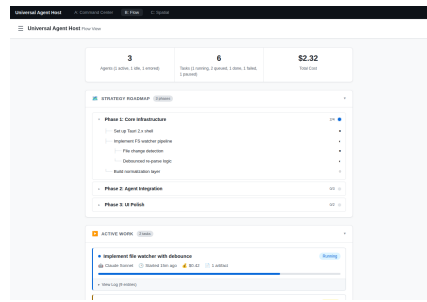
Nerve Center



Mosaic



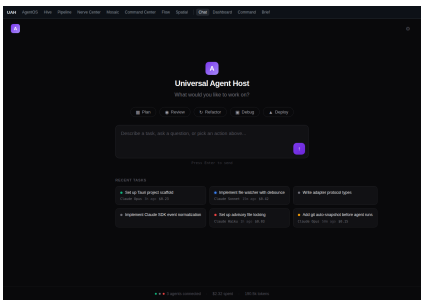
Command Center



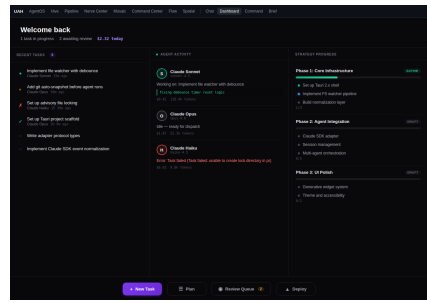
Flow



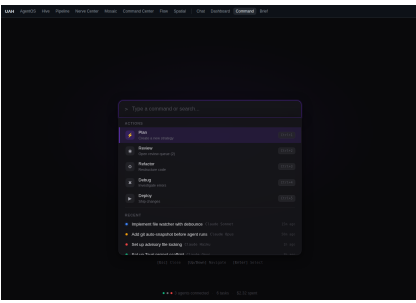
Spatial



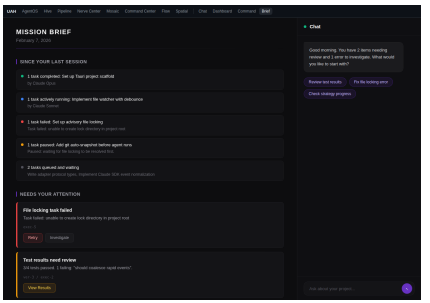
Chat



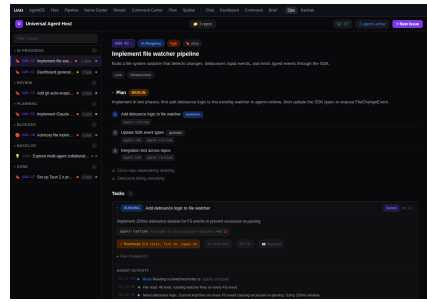
Dashboard



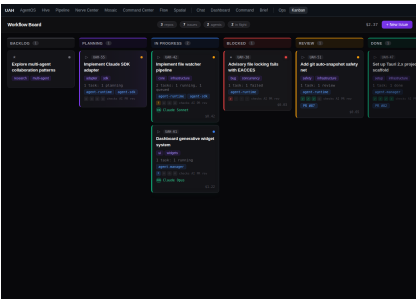
Command



Brief



Ops



Kanban

Appendix C: Verification Pipeline Deep Dive

The workflow variants (Ops and Kanban) introduce a 4-stage verification pipeline that models how code changes are validated in real development workflows.

The 4 Stages

Stage	Type	Description	Gate Criteria
1. Prechecks	Automatic	Lint, types, tests, and build bundled as one gate	All checks pass, tests green, build succeeds
2. AI Review	Automatic	An AI agent reviews the diff for correctness and architecture	No critical findings, severity $\leq$ minor
3. PR	Automatic	Branch pushed, pull request created on the target repo	PR created and CI passes
4. Approval	Manual	Human reviews the PR and approves or requests changes	Required approvals met, no changes requested

Why 4 Stages?

Earlier iterations had 7 granular stages (lint, typecheck, test, build, security scan, PR create, PR review, manual approval). This was refined down to 4 meaningful gates:

- **Prechecks** bundles all automated quality checks. Individually, lint/types/tests/build are implementation details — what matters is "does the code meet quality standards?"
- **AI Review** is the novel addition: an AI agent reviews the diff for correctness, architecture concerns, and potential issues. This catches problems before human reviewers see them.
- **PR** makes the transition from "code on a branch" to "code visible to the team" explicit. This is where the work becomes social and visible.
- **Approval** is the final human gate. After prechecks pass, AI reviews it, and a PR is created, a human makes the final judgment. This keeps humans in the loop for the decisions that matter.

Verification Pipeline in Ops



The 4-stage pipeline visualization in the Ops variant, showing stage status and metadata.

Data Model Evolution

The project uses two data models in parallel. The **original model** (primitives.ts) uses abstract concepts: Context, Strategy, Execution, Verification. The **workflow model** (workflow.ts) uses concrete dev concepts: Project, Issue, Plan, Task, VerificationPipeline. The workflow model was designed to "bridge the gap to a real dev workflow" by using terminology and structures that map directly to how development teams actually think about their work.

Original Model (8 main + 4 startup variants):
Context → Strategy → Execution → Verification

Workflow Model (Ops + Kanban):

Project → Issue (Jira/DevOps) → Plan → Task → VerificationPipeline (4 stages)